CODING FOR DATA SCIENCE AND ANALYTICS

Exercise Session 1

Exercise Sessions

- Exercise sessions take place on **Tuesday, 14:30 - 16:30** (usually; always check the calendar)
- The last session will be dedicated to **exam preparation** and **reviewing topics**.
- If you have questions or need clarification, I am also available:
  - after the exercise sessions;
  - or by arranging another time.
- **d.fracassi002@unibs.it**

Course Materials

All slides, code examples, and exercise solutions will be available on both the **e-learning platform** and **GitHub**.

GitHub Repository:

github.com/Dieg0-f/Coding-for-Data-Science-Lab



Exercise 1 — Swapping Variables

Consider the following variables:

```
x = 2
y = 5
```

Goal

Swap the values of the two variables so that:

```
x = 5
y = 2
```

Question

How can we swap the values of two variables in Python?

Exercise 2 — Temperature Conversion

Start from a temperature expressed in degrees Celsius:

```
1 celsius = 25
```

Goal

Convert the temperature from Celsius to Kelvin and Fahrenheit
Store the two results in two variables:

`kelvin`, `fahrenheit`

Remember:

$$T_K = T_C + 273.15$$

$$T_F = \frac{9}{5} T_C + 32$$

Exercise 3 — Time Conversion

Consider:

5920 seconds

Goal

Convert the total number of seconds into:

hours : minutes : seconds

Store the three results in three variables:

`hours`, `minutes`, `seconds`

Hint

Which Python operators can be used to compute the quotient and the remainder of a division?

The `print()` Function

Until now, we have created and modified variables.
To display something on the screen, Python provides the `print()` function.

For example:

```
1 print("Hello!")
```

produces:

```
Hello!
```

The general structure is:

```
1 print(...)
```

Whatever we put inside the parentheses is passed to the `print()` function.

Printing Text and Variables

There is an important difference between **text** and **variables**.

To print text, we use quotation marks:

```
1 print("temperature")
```

Output:

```
temperature
```

To print the value stored in a variable, we do not use quotation marks:

```
1 temperature = 25
2
3 print(temperature)
```

Output:

```
25
```

Text or Variable?

Consider:

```
1 age = 20
```

What is the difference between these two instructions?

```
1 print(age)
```

and

```
1 print("age")
```

They produce different outputs:

20
age

Remember

Quotation marks tell Python that we are working with **text**, not with a variable name.

Exercise 4 — Your First `print()`

Create a variable called `name` containing your name.

For example:

```
1 name = "Marioa"
```

Then use the `print()` function to display the value stored in the variable.

Exercise 5 — Printing a Number

Create a variable:

```
temperature = 23
```

Use `print()` to display the value stored in `temperature`.

Then change the value of the variable:

```
temperature = 28
```

and run the same `print()` instruction again.

Question

Does the `print()` instruction need to change?

Printing Multiple Objects

The `print()` function can receive more than one object.

For example:

```
1 name = "Alice"
2 age = 20
3
4 print(name, age)
```

Output:

```
Alice 20
```

By default, Python inserts a space between the different objects.

The sep Argument

The separator between objects can be changed using sep.

For example:

```
print("2026", "10", "04", sep="-")
```

Output:

2026-10-04

Another example:

```
print("Python", "R", "SQL", sep=" | ")
```

Output:

Python | R | SQL

Special Characters in Strings

The backslash character

\

has a special meaning inside Python strings.

It can be combined with another character to create an **escape sequence**.

Some common examples are:

Sequence	Meaning
`\n`	new line
`\t`	tab
`\\`	backslash
`\"`	double quotation mark
`\'`	single quotation mark

New Lines with \n

The sequence \n creates a new line.

Example:

```
1 print("Hello\nWorld")
```

Output:

```
Hello
World
```

We can use several \n characters inside the same string.

```
1 print("Line 1\nLine 2\nLine 3")
```

Printing a Backslash

Since the backslash has a special meaning in Python strings, we need two backslashes to represent one actual backslash.

For example:

```
1 print("\\")
```

Output:

```
\
```

Another example:

```
1 print("C:\\Users\\Alice")
```

Output:

```
C:\Users\Alice
```

Quotation Marks Inside Strings

Suppose we want to print:

```
She said "Hello!"
```

One possibility is:

```
1 print('She said "Hello!"')
```

Alternatively, we can escape the quotation marks:

```
1 print("She said \"Hello!\"")
```

Both instructions produce the same output.

Triple-Quoted Strings

Python also allows strings to be written using **triple quotation marks**.

For example:

```
1 text = """First line
2 Second line
3 Third line"""
4
5 print(text)
```

Output:

```
First line
Second line
Third line
```

Triple-quoted strings are useful when a string spans multiple lines.

Exercise 6 — Print the Cat

Write a Python program that uses only **one `print()` statement** to display the following cat:

```
 /\_/\
( o.o )
 > " <
```

Constraint

You can use only one call to `print()`. Make with both single and triple quotation.

Exercise 6 — Hints

Think about the special characters we have just introduced.

In particular:

- How can you move to a new line?
- How can you include a backslash inside a string?
- Could triple quotation marks make the problem easier?

Combining Text and Variables

Suppose we have:

```
1 name = "Alice"
2 age = 20
```

We could write:

```
1 print("Name:", name, "Age:", age)
```

Python provides another convenient way to combine text and variables:

f-strings

F-Strings

An f-string is a string preceded by the letter `f`.

Variables can be inserted directly inside curly brackets:

```
1 name = "Alice"
2 age = 20
3
4 print(f"My name is {name} and I am {age} years old.")
```

Output:

```
My name is Alice and I am 20 years old.
```

General structure

```
1 print(f"Some text {variable}")
```

Expressions Inside F-Strings

Curly brackets can also contain expressions.

For example:

```
1 x = 5
2 y = 3
3
4 print(f"The sum is {x + y}")
```

Output:

```
The sum is 8
```

Therefore, f-strings can include:

- variables;
- arithmetic expressions;
- other Python expressions.

Exercise 7 — Using F-Strings

Create the following variables:

```
name = "Alice"
age = 21
course = "Data Science"
```

Using **one `print()` statement** and an **f-string**, display:

`Alice is 21 years old and studies Data Science.`

Constraint

Do not manually write the values `Alice`, `21`, and `Data Science` inside the final string. Use the variables.

Exercise 8 — Temperature Report

Consider:

```
celsius = 25
```

Calculate the corresponding temperatures in Kelvin and Fahrenheit.

Then use an f-string to display a sentence similar to:

`25 Celsius = 298.15 Kelvin = 77.0 Fahrenheit`

Remember:

$$T_K = T_C + 273.15$$

$$T_F = \frac{9}{5} T_C + 32$$

Summary

Today we saw:

1. variables and assignment;
2. arithmetic operators;
3. integer division and remainder;
4. unit conversions;
5. the `print()` function;
6. printing text and variables;
7. the `sep` argument;
8. escape sequences;
9. `\n` and backslashes;
10. triple-quoted strings;
11. f-strings.